

ANALYSIS OF CARD-SHUFFLE STREAM CIPHER:
CYCLE STRUCTURE, ROTATION VARIANTS AND WEAK KEYS

by

Scott Daniel Russell, Captain, USAF

A Creative Component submitted to the graduate faculty
in partial fulfillment of the requirements for the degree of

MASTER OF SCIENCE

Major: Cybersecurity

Academic Plan Committee:
Dr. Thomas E. Daniels, Major Professor

The student author, whose presentation of the scholarship herein was approved by the academic plan committee, is solely responsible for the content of this work. The Graduate College will ensure this creative component is globally accessible and will not permit alterations after a degree is conferred.

Iowa State University
Ames, Iowa
2026

Copyright © Scott Daniel Russell, 2026. All rights reserved.

TABLE OF CONTENTS

	Page
TABLE OF CONTENTS.....	ii
LIST OF FIGURES.....	iv
LIST OF TABLES.....	v
NOMENCLATURE.....	vi
ACKNOWLEDGMENTS.....	ix
ABSTRACT.....	x
CHAPTER 1. INTRODUCTION.....	1
Motivation.....	1
Why This Project.....	2
Approach and Scope.....	3
Contributions.....	4
What This Work Does Not Claim.....	4
Organization of This Work.....	5
CHAPTER 2. BACKGROUND AND RELATED WORK.....	6
Solitaire.....	6
How It Works.....	6
Known Analysis.....	6
Relation to This Work.....	7
Mirdek.....	8
How It Works.....	8
Plaintext Dependence, and Why It Matters Here.....	8
Rivest Cipher 4 (RC4).....	9
How It Works.....	9
Known Weaknesses.....	10
Relation to This Work.....	10
Other Hand Ciphers.....	11
CHAPTER 3. THE DANIELS' CIPHER AND ITS VARIANTS.....	13
Deck Structure and Notation.....	13
The Two Shuffle Operations.....	13
The Count Cut.....	13
The Search Cut.....	14
The Turn and the Round.....	14
Pseudocode.....	15
The Key Schedule.....	15
A Worked Example.....	16
The Rotation Variants.....	20

Keystream Output Rate.....	21
CHAPTER 4. APPROACH AND ANALYSIS OF THE STATE SPACE.....	24
The State Space.....	24
Note on Key Space.....	26
The Round Map Is One-to-One.....	26
Three Methods.....	27
Correctness and Validation.....	28
Cycle Structure Results.....	29
Results in Bits.....	32
CHAPTER 5. ANALYSIS OF KEYING.....	35
The Key Space and the State Space.....	35
Keys Are Distributed in Proportion to Cycle Length.....	35
Weak Keys.....	38
Expected Cycle Length and Entropy.....	39
Choosing a Rotation.....	41
CHAPTER 6. CONCLUSIONS AND FUTURE WORK.....	42
Summary of Findings.....	42
Limitations.....	42
Future Work.....	43
Closing.....	45
REFERENCES.....	46
APPENDIX. SOFTWARE AND DATA.....	48

LIST OF FIGURES

	Page
Figure 1. Keystream output rate per round as a function of deck size. The rate grows as $(n / 2) \log_2 n$, so the full-scale 26-card cipher emits roughly five times as many bits per round as the deck-8 variant analyzed here.....	23
Figure 2. Cycle-length spectrum at rotation 0 for deck sizes 4, 6 and 8, sorted longest first on a logarithmic scale. Grey bars mark cycles that no key's starting state lies on, so no key can ever enter them. The distribution is consistently top-heavy: a small number of very large cycles account for almost the entire state space, followed by a long tail of tiny cycles.....	30
Figure 3. Deck size 8: cycle-length spectrum under each discard rotation, on a logarithmic scale. The rotation is not a cosmetic parameter. It changes the number of cycles from 18 to 28 and the share of the state space held by the largest cycle from 44.83 to 78.40 percent.....	32
Figure 4. Keystream period in bits before repetition, by deck size and rotation, on a logarithmic scale. Solid bars give the expected period for a key drawn uniformly at random, weighted by the number of keys landing on each cycle; the paler bars beside them give the longest cycle available. The two are close at every deck size, because most keys land on the dominant cycle. 33	
Figure 5. Share of the key space landing on each cycle, plotted against that cycle's length, on logarithmic axes. Points fall along a line of slope one, showing that keys are distributed in proportion to cycle length. The dashed line marks the weak-key threshold $n!$, below which a cycle is considered cryptographically inadequate.....	37

LIST OF TABLES

	Page
Table 1. Keystream output rate by deck size. Each turn emits exactly one output, so the turns and outputs columns carry the same value.....	22
Table 2. State space and key space by deck size.....	25
Table 3. Complete cycle decompositions for deck sizes 4 and 6 (true periods in rounds, longest first). An asterisk marks a cycle that no key's starting state lies on: the key schedule never produces a state on this cycle, so no key can enter it.....	31
Table 4. Deck size 8 cycle structure by rotation. The state space is $1,625,702,400 = (8!)^2$ states in every case.....	31
Table 5. Keystream period before repetition, in rounds and in bits, by deck size and rotation. Expected values are weighted by the number of keys landing on each cycle.....	33
Table 6. Weak keys by deck size and rotation, using the threshold $n!$	38
Table 7. Keying statistics by deck size and rotation. Entropy is measured over the distribution of keys across cycles.....	40
<i>Table 8. The Daniels' Cipher - Deck Size 8: Rotation 0 full decomposition.....</i>	<i>49</i>
<i>Table 9. The Daniels' Cipher - Deck Size 8: Rotation 1 full decomposition.....</i>	<i>50</i>
<i>Table 10. The Daniels' Cipher - Deck Size 8: Rotation 3 full decomposition.....</i>	<i>51</i>
<i>Table 11. The Daniels' Cipher - Deck Size 8: Rotation 5 full decomposition.....</i>	<i>52</i>

NOMENCLATURE

Boundary state	The ordered pair of decks at a round boundary. C_0 denotes the arrangement the key schedule produces; C_1 , one round later, is the first boundary state a key reaches
CBC	Cipher block chaining
CC	Creative Component
Count cut	A rotation of the search deck by the value of a drive card
Cycle	The sequence of boundary states that repeated rounds visit until the starting state recurs. Every state lies on exactly one cycle, and its length, in rounds, is how long the cipher runs before its state repeats
Discard pile	Cards spent from the drive deck during a round, each placed at the bottom, so the first card spent is on top
Drive deck	The deck of n cards consumed two cards per turn to drive the shuffles
F	The macro map: the state transition induced by one round
Full round	The original implementation's name for what this work calls a round
ISU	Iowa State University
IV	Initialization vector
Key	The secret input string. Wrapped by the IV and run through the key schedule, it places the two decks in their starting arrangement
Key sequence	The string the key schedule actually consumes: the IV, followed by the key, followed by the IV again. Every run reported here uses an empty IV, so the key sequence is the key itself, exactly n characters

Keystream	The sequence of output cards the cipher emits, one per turn. An output of the cipher, not an input to it
Keystream card	A single output card, read from the top of the search deck at the end of one turn. A round emits $n/2$ of them
KSA	Key-scheduling algorithm (RC4)
LC4	ElsieFour, Kaminsky's 2017 low-tech authenticated cipher
Lehmer code	The rank of a permutation in lexicographic order, 0 to $n! - 1$
n	The size of one deck. The cipher uses two decks of n cards, so a deck size of 8 means sixteen cards in total
PRGA	Pseudo-random generation algorithm (RC4)
RC4	Rivest Cipher 4, the software stream cipher whose structure this work compares against
Rotation r	The fixed count cut applied to the discard pile at each round boundary
Round	The $n/2$ turns after which the drive deck is exhausted and the decks are exchanged; the unit in which every cycle length and period here is counted. A round emits $n/2$ keystream outputs. The original implementation calls it a full round
Search cut	A deal-and-reassemble of the search deck around a target card
Search deck	The deck of n cards that is shuffled and whose top card, after each turn, is output into the keystream
Turn	One count cut and one search cut. Consumes the bottom card and then the top card of the drive deck, and emits one keystream card. $n/2$ turns make one round. The original implementation calls it a round
USAF	United States Air Force

VIC	A 1950s Soviet field cipher, named for the codename of the agent who carried it
Weak key	A key whose first boundary state lies on a cycle shorter than $n!$

ACKNOWLEDGMENTS

I would like to thank Dr. Thomas Daniels, who designed the cipher studied here and for the weekly discussions that shaped this research. Several of the results in this work exist because Dr. Daniels asked the right question at the right time: the suggestion that the discard rotation be treated as an experimental variable in its own right produced Chapter 4, and his insistence that the keystream be measured in bits per round rather than in rounds produced the comparison in Chapter 3 that lets the deck sizes be compared directly.

I would also like to thank my wife, Sydney. Her devotion to our family has made it possible for me to balance active duty service, this degree, and still be the husband and father I want to be. Her support carried our family through this, and without it, pursuing this degree would not have been possible.

The views expressed in this work are those of the author and do not reflect the official policy or position of the United States Air Force, the Department of Defense, or the United States Government.

ABSTRACT

Hand-executable ciphers occupy an unusual position in cryptography: they must be simple enough to run with physical objects, yet they are asked to resist adversaries with substantial computational resources. The best-known modern example, Bruce Schneier's Solitaire, developed for the book *Cryptonomicon* (written by Neal Stephenson), was shown by Paul Crowley to have a measurable bias in its output, and the analysis of such ciphers has generally been statistical.

This work takes the opposite approach. It studies a new card-shuffle stream cipher designed by Dr. Thomas Daniels, one that does not appear in the published literature and has not been analyzed before, in which the keystream depends only on the key and initialization vector and not on the plaintext. In my research, I evaluate this cipher at reduced deck sizes where we can practically observe the entire state space. The state is the ordered pair of two n -card decks, giving a state space of $(n!)^2$, and the transition induced by one round is shown to be one-to-one, so the state space splits cleanly into separate cycles. For deck sizes 4, 6 and 8, and for four values of the discard rotation parameter, every cycle is listed and every key is mapped to the cycle it lands on. At deck size 8 this means resolving all 1,625,702,400 $(8!)^2$ states and all 16,777,216 (8^8) keys, which was achieved on a machine with under 2 GB of usable memory by storing one bit per state and recomputing the transition on demand.

The results show that the discard rotation, a parameter with no obvious cryptographic significance, changes the cycle structure dramatically: at deck size 8 the share of keys landing on the largest cycle ranges from 44.83% to 78.40% depending on the rotation. Keys are shown to be distributed across cycles in proportion to cycle length, with no measurable bias. This work is an exploration of The Daniels' Cipher exploring deck sizes 4, 6, and 8.

CHAPTER 1. INTRODUCTION

Motivation

A hand cipher is one that a person can execute with physical objects and no computing equipment. Hand ciphers are an old category, but the need for them has not gone away. An agent using one has to be able to encrypt and decrypt messages without carrying anything that would incriminate them, and the method has to be simple enough to use even if the agent has little knowledge in mathematics or cryptography. A hand cipher today has to be simple enough to use correctly under pressure, while still holding up against an adversary with significant computing power.

That tension has produced a literature of serious designs and a smaller literature of serious attacks on them. Bruce Schneier's *Solitaire*, published as an appendix to Neal Stephenson's novel *Cryptonomicon*, is the best-known example, and Paul Crowley's analysis of it is the best-known attack: successive keystream outputs collide noticeably more often than chance permits. Crowley then designed his own card cipher, *Mirdek*, to avoid *Solitaire*'s problems, and withdrew it within a year as insecure against chosen plaintext. Weaknesses in these hand ciphers tend to be structural rather than a matter of key size.

A second difficulty is computational. The state space of a 54-card cipher is vastly large, so published analyses are necessarily statistical: biases are estimated from samples, and cycle lengths are also only estimated. Daniel Shiu could only argue informally that *Solitaire*'s cycles resemble those of a random one-to-one map, and Wiem Tounsi and colleagues gave a probability-based method for estimating the keystream cycle length. The same limitation applies to the machine ciphers in this structural family. RC4, whose internal state is a permutation of 256 bytes plus two pointers, has a state space on the order of 2^{1700} . No one has listed its cycle

structure or established its period, and its eventual deprecation came from an accumulation of statistical biases rather than from any single structural breakdown.

In short, hand ciphers face two fundamental challenges: they must be simple enough for a person to execute reliably without specialized equipment, yet robust enough to withstand modern cryptanalytic attacks; and their enormous state spaces make exhaustive analysis impractical, forcing researchers to rely heavily on statistical evidence and probabilistic estimates.

Existing work on Solitaire, Mirdek, and RC4 demonstrates that structural weaknesses can emerge even in ciphers with extremely large state spaces, but it also shows how difficult it can be to characterize those weaknesses completely. This motivates a closer examination of the structural properties of card-based ciphers, particularly their state spaces, keystream generation, and cycle behavior, with the goal of determining what can be established analytically rather than inferred solely from statistical experimentation.

Why This Project

I came to this problem as an active duty Air Force Captain in the Offensive Cyberspace Operations career field. I am currently assigned to teach Air Force ROTC at Iowa State University, and I will likely return to the operational cyberspace community after this instructor tour. Completing this degree in Cybersecurity supports that career field directly and factors into my future desire to teach at the United States Air Force Academy. That background shaped this project in two ways worth stating, because they explain both the topic and how the results are reported.

The first is that low-technology methods still matter to national security. Secure communication does not always come with a power supply, a trusted device, and a network. People who operate in denied or degraded environments, and the national intelligence services

that have historically relied on field ciphers, need methods that work with objects a person can carry, use under pressure, and explain to someone else. The VIC cipher, a 1950s Soviet field cipher discussed in Chapter 2, is the clearest example: a hand method, carried by a single agent, that resisted a national code-breaking effort for years. A deck of cards is not a curiosity in that setting. It is a communications capability that blends in cleanly, and understanding how strong such a method really is has a direct bearing on whether it should ever be trusted in an operational environment.

The second is a mentality learned from my time conducting offensive operations: looking for the flaw before an adversary finds it, and treating a system as weak until someone has genuinely tried to break it. A design is only as trustworthy as the effort that has been spent trying to defeat it, and for this cipher that effort had not yet been spent: it is new, and nobody had measured it. That is the gap this work sets out to close. Knowing exactly how long the keystream runs before it repeats, and exactly which keys land on short cycles, is the first thing an operator would need to know before trusting a method like this in the field.

Approach and Scope

This work takes a different route to the same questions. Rather than sampling a cipher too large to analyze in full, it studies the cipher at deck sizes small enough to analyze completely, and then asks what the exact structure looks like and how it changes as the deck grows. The cipher is a stream cipher designed by Dr. Thomas Daniels, built from the same physical primitives as Mirdek, the count cut and the search cut, but arranged so that the state evolves from the key and initialization vector alone. That single design decision, plaintext independence, is what makes the analysis possible: the keystream is a function of the key, so the cipher is a deterministic system.

This work analyzes several variants of The Daniels' Cipher. The variants are the deck size n , taken as 4, 6 and 8, and the discard rotation r , taken as 0, 1, 3 and 5. For each of the twelve combinations the entire state space of $(n!)^2$ states is decomposed into cycles, and the entire key space of n^n keys is mapped onto those cycles. Nothing is sampled and nothing is estimated; the cycle lengths reported here are exact for those deck sizes of 4, 6, and 8.

Contributions

The specific contributions of this work are as follows:

- An initial analysis and description of The Daniels' Cipher in detail.
- A demonstration that the cipher's round transition is one-to-one on the $(n!)^2$ state space, so the space splits into separate cycles with no dead ends. This is what makes listing every cycle meaningful, and it distinguishes the cipher from Solitaire, whose update is known not to be reversible.
- A complete cycle breakdown for deck sizes 4, 6 and 8 at four rotation values, including cycles that no key's starting state lies on, verified by the requirement that cycle lengths sum exactly to $(n!)^2$ in all twelve cases.
- A method that makes the deck-8 case parsable: storing one bit per state and recomputing the transition on demand reduces the working set from roughly 28 GB to under 400 MB, allowing 1.6 billion states to be resolved on a modest system with under 2 GB of usable memory.
- The finding that the discard rotation parameter substantially influences the cycle structure.
- A measurement of how the key schedule distributes keys across the state space, showing that keys land on cycles in proportion to cycle length with no detectable bias based on statistical analysis.
- A common unit, keystream bits before repetition, that makes the deck-size variants comparable to one another and to other currently published stream ciphers.

What This Work Does Not Claim

This is an exploration of a cipher's structure, not a security proof, and that distinction matters. Mapping out the cycle structure establishes how long the keystream can be before it repeats and how the key space distributes over those cycles. It does not establish that the cipher resists known-plaintext or chosen-plaintext attack, that the key or state can be recovered, or that structural shortcut exists. No such analysis has been performed here. One further limitation is that the results are for deck sizes 4, 6 and 8 given our modest computational limits.

Organization of This Work

Chapter 2 reviews the prior work this cipher descends from, covering Solitaire, Mirdek and RC4. Chapter 3 describes The Daniels' Cipher itself: the deck structure, the count cut and search cut in plain language and in pseudocode, and how fast the cipher produces output.

Chapter 4 sets out the state encoding and the three successive analysis methods used, the correctness problems found along the way, and the cycle-structure results. Chapter 5 analyzes the keying: how the key space maps into the state space, how keys spread across cycles, and what proportion of keys are weak. Chapter 6 draws conclusions and sets out future work.

CHAPTER 2. BACKGROUND AND RELATED WORK

This chapter covers the three ciphers that are most relevant. Solitaire establishes the design problem and the standard against which hand ciphers are measured; Mirdek supplies the physical primitives this cipher is built from and RC4 supplies the structural template. A shorter section discusses other related hand ciphers.

Solitaire

How It Works

Solitaire was designed by Bruce Schneier in 1999 and published alongside Neal Stephenson's novel *Cryptonomicon*, where it appears as *Pontifex*. Solitaire operates on a 54-card deck: the standard 52 cards plus two jokers that can be told apart. Cards take values 1 through 53 by bridge suit order, with both jokers valued 53. The initial ordering of the deck is the key.

Generating each keystream card takes five steps. Joker A moves down one position and joker B moves down two, each wrapping at the bottom of the deck. A triple cut then exchanges the block of cards above the top joker with the block below the bottom joker. A count cut follows, using the value of the bottom card to cut that many cards from the top and reinsert them just above the bottom card. Finally the value of the top card is read and used to count down into the deck; the next card is the output. If that card is a joker, nothing is output and the process repeats. Output cards convert to the range 1 to 26 and combine with the plaintext by addition mod 26.

Known Analysis

Crowley's analysis is the central result. He observed that successive keystream outputs appear substantially more often than the one-in-26 rate that a uniform keystream would give, reporting from experiments on ten million samples that the observed frequency is closer to one in

22.5. The conditional form of the bias is far stronger: when the top card takes the same value in two successive iterations, an event of roughly two percent probability, the two output letters agree about 34 percent of the time.

Crowley reported a second finding with direct bearing on the present work: Solitaire's state update is not reversible. The initial step, in which a joker on the bottom of the deck moves to the top, cannot be undone, so two distinct deck configurations can share a successor. Solitaire's update is therefore a general function on its state space rather than a permutation of it.

Later work refined the picture. Shiu built a model accounting for most of the repetition bias and argued that the state update is unlikely to produce cycles significantly shorter than a random one-to-one map would. Tounsi and colleagues gave a probability-based method for estimating the cycle length of the Solitaire keystream, and evaluated a modified variant against the National Institute of Standards and Technology (NIST) statistical test suite.

Relation to This Work

Solitaire is the direct ancestor in design intent: a keystream generator whose state is a deck of cards, whose operations are shuffles a human can perform, and whose output combines with plaintext mod 26. The cipher studied here shares all of that.

There are two distinct differences. The first is being one-to-one. Because Solitaire's update is not one-to-one, its state graph contains lead-in paths feeding into loops, and parsing for Solitaire's full cycle decomposition is challenging. Chapter 4 shows that The Daniels' Cipher has a round transition that is one-to-one, so its state space splits into separate cycles with no dead ends at all, and the listing is exact. The second is precision. Shiu could argue about Solitaire's cycle structure and Tounsi and colleagues could estimate it, because 54 cards put a full listing out of reach. At deck size 8 the present work computes the decomposition outright. That is the trade

this project makes: a smaller cipher, studied exactly, instead of a full-size cipher studied approximately.

Mirdek

How It Works

Mirdek is Paul Crowley's own card cipher, designed after his analysis of Solitaire and with the same goal: strong hand-operable encryption. It uses a standard 52-card deck with the jokers removed, mapping black aces through kings to A through M and red aces through kings to N through Z. A notable practical choice is that Mirdek avoids mental arithmetic entirely; the operator counts and deals rather than adding mod 26.

Two of Mirdek's primitives are the ones this cipher inherits. The counted cut takes the top card of one pile, places it face up on a discard pile, and counts that many cards from the top of the other pile to its bottom. The letter search deals cards from the top of a pile onto two alternating piles until the targeted card appears, then reassembles the piles in a prescribed order. Operation proceeds through initialization, keying from a passphrase, mixing, and encryption.

Plaintext Dependence, and Why It Matters Here

Mirdek's encryption step performs a letter search for the next unused letter of the plaintext. The plaintext therefore determines which card is searched for, and the search itself permutes the deck, so the state path depends on the message. Crowley lists this as a deliberate feature, describing plaintext feedback rather than pure generator operation as making analysis harder. Mirdek is consequently not a stream cipher in the strict sense: there is no keystream that exists independently of the message, and the construction is closer in spirit to cipher block chaining (CBC).

The intuition did not survive contact with analysis. In an update from January of 2000, Crowley withdrew the cipher, stating that it is not secure and is terrifically weak against a chosen plaintext attack.

This explains one of the reasons for the design choices of The Daniels' Cipher studied here. Where Mirdek mixes plaintext into the state, this cipher evolves its state from the key and initialization vector alone. The security argument for that choice is Crowley's own experience. The analytical argument is that plaintext independence is precisely what makes this work possible: because the keystream depends only on the key, the cipher defines a fixed map on a fixed state space, its loops are well defined, and listing them answers a real question. A plaintext-dependent cipher has no key-only path to trace, and the analysis in Chapters 4 and 5 would have no bearing.

Rivest Cipher 4 (RC4)

How It Works

RC4 was designed by Ron Rivest in 1987 and held as a trade secret until it was leaked in 1994. Its internal state is a permutation S of the 256 byte values together with two 8-bit pointers i and j . The key-scheduling algorithm initializes S to the identity and then makes 256 passes, at each step advancing j by the current state byte and a key byte and swapping two entries. The generation algorithm then repeatedly advances i by one, advances j by $S[i]$, swaps $S[i]$ and $S[j]$, and outputs the byte found at position $S[i] + S[j]$.

The structural resemblance to a card cipher is close enough that it has been noticed before. Matthijs van Duin described performing RC4 by hand with a physical deck in 2001, using a reduced deck of 26 or 52 cards and two distinct markers to hold the positions of i and j , and McKague's 2005 master's thesis presents playing-card implementations of RC4-like ciphers.

RC4 can therefore be described directly as a card cipher: a deck of cards, two markers, a swap and an output per step. A faithful 256-card version with two coins as markers is a useful way to see the equivalence, though, as the published versions acknowledge by shrinking the deck, a 256-card deck is a thought experiment rather than a practical hand cipher.

Known Weaknesses

RC4's failure was cumulative rather than sudden, and the pattern is a useful checklist for any cipher in this family. Fluhrer and McGrew found biases in the probabilities of output pairs; Mantin and Shamir showed the second output byte is biased toward zero; Fluhrer, Mantin and Shamir found a large class of weak keys and broke the mode of operation used by Wired Equivalent Privacy (WEP); AlFardan and colleagues turned the accumulated biases into plaintext recovery against RC4 in TLS; and Vanhoef and Piessens demonstrated practical cookie decryption. RC4 was formally banned in Transport Layer Security (TLS) by a specific Internet Engineering Task Force (IETF) standards document (Request For Comments (RFC) 7465), which was authored by Andrei Popov in 2015.

One further category matters directly here. A colliding key pair is a pair of distinct keys that produce the same internal state after key scheduling, and therefore the same keystream. Matsui found a 24-byte colliding pair in 2009, Chen and Miyaji reduced it to 22 bytes in 2011, and Jana and Paul later found more. Seventeen years of dedicated search shortened the known colliding key by two bytes and never settled whether shorter ones exist, because the search space is too large to work through.

Relation to This Work

RC4 is the structural ancestor of the cipher studied here: both keep a permutation as state, advance it by swaps and cuts driven by the state itself, and emit one symbol per step. The

contrast is the point of this project. RC4's state space of $256! \times 256^2$, on the order of 2^{1700} , puts every question about its global structure permanently out of reach. Its period is unknown and computationally infeasible to compute, and its biases and key collisions are found by sampling and search. RC4 was abandoned because of accumulated statistical evidence.

At deck size 8, The Daniels' Cipher studied here has 1,625,702,400 (1.6 billion) states and 16,777,216 (16.8 million) keys. Both fit in memory, and the questions that can only be sampled for RC4 can be answered outright: the exact cycle decomposition, the exact shortest cycle any key can reach, and the exact number of keys landing on each cycle. The framing of this work is therefore that it asks the RC4 questions on a family of ciphers small enough to answer them exactly, and then examines how the answers move as the deck grows. The keying analysis in Chapter 5 is the direct analogue of the RC4 key collision literature, and the future work in Chapter 6 identifies the colliding-key question as the natural next step.

Other Hand Ciphers

Three further designs are worth brief mention for context.

The VIC cipher, named after the codename "VICTOR" of the Soviet agent Reino Häyhänen who used it, was employed in the 1950s and reconstructed after his defection in 1957. It combines a straddling checkerboard, a lagged Fibonacci generator, chain addition and a disrupted double columnar transposition. It is the benchmark for how much complexity a human operator will tolerate in the field, and evidence that hand ciphers can defeat state adversaries in practice.

The Chaocipher, invented by John F. Byrne in 1918 and kept secret until his family donated the papers to the National Cryptologic Museum in 2010, with Rubin (2011) giving the technical disclosure, uses two rotating alphabets and permutes both after every character. It is the

clearest historical precedent for the central idea of the present cipher, a state that is permuted after every output symbol.

LC4, published by Kaminsky in 2017, is the modern state of the art, and offers two useful contrasts: it is plaintext-dependent, placing it on the same branch as Mirdek and the opposite branch from this work, and it provides authentication, which the cipher studied here does not. The Daniels' Cipher here provides confidentiality only, and any deployment would need a separate integrity mechanism.

CHAPTER 3. THE DANIELS' CIPHER AND ITS VARIANTS

Deck Structure and Notation

The Daniels' Cipher operates on $2n$ cards divided into two decks of n cards each, together with a discard pile that begins empty. Throughout this work n denotes the size of one deck, so a deck size of 8 means two decks of eight cards and sixteen cards in total. The full-scale cipher is defined for $n = 26$, one half of a standard deck per side.

- The drive deck supplies the values that drive the shuffles. It is consumed two cards per turn: its bottom card, then its top card.
- The search deck is the deck that gets shuffled, and its top card after each turn is the keystream output.
- The discard pile accumulates the cards spent from the drive deck, each placed at the bottom so that the first card spent stays on top, and becomes the next search deck when the drive deck is exhausted.

Cards are represented as integers 0 through $n - 1$, and a deck written out as a sequence, such as A 2 3 4, lists its cards from top to bottom. The cipher requires n to be even. The search cut deals cards alternately onto two piles, and the reassembly rule depends on the deck having an even number of cards. All experimental configurations conducted preserve this even deck requirement.

The Two Shuffle Operations

The Count Cut

The count cut is a rotation. Given a count c , the top c cards of the search deck are moved to the bottom, one at a time, preserving their order. It is the lighter of the two operations and contributes little diffusion on its own; its role is to displace the deck by a data-dependent amount before the heavier operation runs. A count of zero is a no operation (no-op), and this special case turns out to matter: a key that repeatedly places low-valued cards at the driving position weakens the contribution of this step, which is one mechanism by which short cycles can arise.

The Search Cut

The search cut is where the diffusion happens. Given a target card value, cards are dealt one at a time from the top of the search deck, face up, onto two piles alternately, until the target card is dealt. The pile that received the target card is placed on top of the other pile, and the combined piles are placed underneath the cards that were never dealt. The reassembled deck is the new search deck, and its top card is the turn's output into the keystream.

The operation is worth describing in physical terms because its effect is easier to see than to read. Dealing alternately reverses the relative order of the dealt cards within each pile and combines two subsequences that were adjacent; the reassembly then moves that whole reversed block beneath the undealt remainder. Where the block lands depends on where the target card happened to fall, which is why the operator cannot anticipate the resulting order partway through, and why the operation mixes better than the count cut.

The Turn and the Round

One turn consumes two cards from the drive deck and produces one output. The bottom card of the drive deck is removed, placed at the bottom of the discard pile, and its value used as the count for a count cut of the search deck. The top card of the drive deck is then removed, placed at the bottom of the discard pile, and its value used as the target for a search cut of the search deck. The top card of the resulting search deck is the keystream output.

Because each turn consumes two of the drive deck's n cards, the drive deck is exhausted after exactly $n / 2$ turns. Those turns make up one round, and the round is the natural unit of the cipher. Because each turn emits one card, a round produces exactly $n / 2$ keystream outputs: two at deck size 4, four at deck size 8, and thirteen at the full-scale 26. At a round boundary three

things happen. The discard pile, which now holds all n cards previously in the drive deck, is count cut by a fixed rotation r . The old search deck becomes the new drive deck, and the rotated discard pile becomes the new search deck. The discard pile is then emptied.

All cycle lengths in this work are counted in rounds, because the round boundary is where the cipher's state is sampled. The keystream output rate, derived later in this chapter, converts these into output symbols and bits. The original implementation's documentation calls the turn a round and the round a full round; this work says turn and round so that the unit in which everything is counted has the plain name.

Pseudocode

```

turn(drive, search, discard):
    if drive is empty:                                # round boundary
        count_cut(discard, r mod n)                  # r = rotation parameter
        drive <- search
        search <- discard
        discard <- empty
        record state (search, drive)

    c <- remove_bottom(drive); add_bottom(discard, c)
    count_cut(search, c)

    t <- remove_top(drive);    add_bottom(discard, t)
    search_cut(search, t)

    return search[0]                                # keystream output

count_cut(deck, c):                                # move top c cards to the bottom, in order
search_cut(deck, t):                                # deal alternately onto piles A and B until
                                                    # t is dealt; place the pile holding t on
                                                    # top of the other; place both beneath the
                                                    # undealt remainder

```

The Key Schedule

Keying is performed before generation begins, and it is what connects a key to a state. The key string is wrapped by the Initialization Vector (IV), so the sequence actually consumed, called the key sequence throughout this work, is the IV followed by the key followed by the IV again. The key is a string of letters rather than of numbers, and throughout this work the key is

exactly n characters long over an n -letter alphabet, which is what makes the key space n^n . Every run reported in this work uses an empty IV, so the key sequence and the key coincide at exactly n characters; varying the IV, and with it the length of the key sequence, is left to future work. Each character is mapped to a card value mod n , and successive characters drive alternating count cuts and search cuts on both decks, starting from both decks in order. Every run reported here begins from that same arrangement, 0 through $n - 1$ in both decks, so with the IV held empty the key is the only variable. The key therefore does not name a state directly; it performs the cipher's own shuffles to arrive at one. The deck pair the schedule leaves behind is written C_0 . Running one round from C_0 gives the key's first boundary state, written C_1 , and everything about that key's period follows from which cycle C_1 lies on. This is the sense in which a key reaches a state, and it is why Chapter 5 treats the mapping from keys to states as an empirical question.

```

keystate(drive, search, key, iv):
  full <- iv + key + iv          # the key sequence: key wrapped by IV
  for i in 0 .. len(full) - 1:
    face      <- value(full[i])
    nextface <- value(full[(i + 1) mod len(full)])
    count_cut (drive,  face)
    search_cut(drive, nextface)
    search_cut(search, face)
    count_cut (search, nextface)
  # both decks now hold this key's starting arrangement, C_0;
  # one round from here produces the key's first
  # boundary state C_1

```

A Worked Example

The two shuffle operations are easier to follow with an actual example than with the description above. The following is a complete run through of two rounds of the deck-size-4 cipher, worked by hand with two small decks of playing cards, using the rotation value $r = 3$.

Card values run Ace = 0, Two = 1, Three = 2, Four = 3. To start, the drive deck holds the four spades in order from top to bottom, A♠ 2♠ 3♠ 4♠, and the search deck holds the four hearts in the same order, A♥ 2♥ 3♥ 4♥; the discard pile starts empty. The suits make the walkthrough

easy to follow by eye and explore a physical card deck use case. As the rounds proceed, the two decks stop lining up with their starting suits, which is itself a useful thing to watch happen.

The first turn of round 1 removes the bottom card of the drive deck, $4\spadesuit$ (value 3), and uses it for a count cut: the top three cards of the search deck move to the bottom, turning $A\heartsuit\ 2\heartsuit\ 3\heartsuit\ 4\heartsuit$ into $4\heartsuit\ A\heartsuit\ 2\heartsuit\ 3\heartsuit$. The $4\spadesuit$ starts the discard pile.

The top card of the drive deck, $A\spadesuit$ (value 0), is removed next and used as the target for a search cut: cards are dealt one at a time off the top of the search deck, alternating between two piles, until a card of value 0 appears. Dealing $4\heartsuit\ A\heartsuit\ 2\heartsuit\ 3\heartsuit$, the first card dealt is $4\heartsuit$, which goes to one pile; the second card dealt is $A\heartsuit$, which has the target value and goes to the other pile, ending the deal. The new search deck is built from the cards left undealt (here, $2\heartsuit\ 3\heartsuit$, still in order), followed by the pile that caught the target read in reverse (just $A\heartsuit$), followed by the other pile read in reverse (just $4\heartsuit$): $2\heartsuit\ 3\heartsuit\ A\heartsuit\ 4\heartsuit$. The top card, $2\heartsuit$, is the round's first keystream output. $A\spadesuit$ goes to the bottom of the discard pile, beneath $4\spadesuit$.

The second turn of round 1 works the same way on what is left of the drive deck, $2\spadesuit$ and $3\spadesuit$. The bottom card, $3\spadesuit$ (value 2), drives a count cut that rotates the search deck by two, turning $2\heartsuit\ 3\heartsuit\ A\heartsuit\ 4\heartsuit$ into $A\heartsuit\ 4\heartsuit\ 2\heartsuit\ 3\heartsuit$. The top card, $2\spadesuit$ (value 1), is the target for a search cut: dealing $A\heartsuit\ 4\heartsuit\ 2\heartsuit\ 3\heartsuit$ alternately, $A\heartsuit$ (value 0) and $4\heartsuit$ (value 3) go to the first and second piles respectively, and then $2\heartsuit$ (value 1) matches the target and joins the first pile, ending the deal. The new search deck is the undealt remainder ($3\heartsuit$), followed by the first pile reversed ($2\heartsuit$ then $A\heartsuit$), followed by the second pile reversed ($4\heartsuit$): $3\heartsuit\ 2\heartsuit\ A\heartsuit\ 4\heartsuit$. The keystream output is $3\heartsuit$. The drive deck is now empty, both $3\spadesuit$ and $2\spadesuit$ have gone to the bottom of the discard pile, and round 1 is complete after two turns, which matches the $n / 2 = 2$ turns per round that Table 1 gives for a four-card deck.

At the first round boundary the discard pile, now holding $4\spadesuit A\spadesuit 3\clubsuit 2\spadesuit$ from top to bottom, in the order the cards were spent, is count cut by the rotation value; at $r = 3$ this moves the top three cards to the bottom, turning $4\spadesuit A\spadesuit 3\clubsuit 2\spadesuit$ into $2\spadesuit 4\spadesuit A\spadesuit 3\clubsuit$. The old search deck becomes the new drive deck, $3\heartsuit 2\heartsuit A\heartsuit 4\heartsuit$, and the rotated discard becomes the new search deck, $2\spadesuit 4\spadesuit A\spadesuit 3\clubsuit$; the discard pile is emptied. Note what happened to the suits: the search deck, which started as all hearts, is now all spades, and the drive deck is now all hearts. One round has completely swapped which deck holds which suit, which is a visible, physical demonstration of the mixing this cipher is meant to provide. This arrangement is the first boundary state C_1 , written (search $2\spadesuit 4\spadesuit A\spadesuit 3\clubsuit$, drive $3\heartsuit 2\heartsuit A\heartsuit 4\heartsuit$), whose state code under the encoding of Chapter 4 is 254.

Round 2 begins with drive deck $3\heartsuit 2\heartsuit A\heartsuit 4\heartsuit$ and search deck $2\spadesuit 4\spadesuit A\spadesuit 3\clubsuit$. The bottom card, $4\heartsuit$ (value 3), drives a count cut that rotates the search deck by three, turning $2\spadesuit 4\spadesuit A\spadesuit 3\clubsuit$ into $3\clubsuit 2\spadesuit 4\spadesuit A\spadesuit$. The top card, $3\heartsuit$ (value 2), is the search target, and this time it is the very first card dealt: $3\clubsuit$ already has value 2, so the deal ends immediately with nothing dealt to the other pile. The new search deck is the undealt remainder, $2\spadesuit 4\spadesuit A\spadesuit$, followed by the pile that caught the target (just $3\clubsuit$), followed by the empty other pile: $2\spadesuit 4\spadesuit A\spadesuit 3\clubsuit$, which is the order the search deck held before the count cut at the start of this turn; the rotation and the search cut happened to cancel. The keystream output is $2\spadesuit$.

The last turn is the one worth pausing on. The bottom card of the drive deck is now $A\heartsuit$, which has value 0, so the count cut rotates the search deck by zero cards, which is to say it does nothing at all; this is the no-op case noted earlier, where a low-valued card leaves the count cut without any effect. The top card, $2\heartsuit$ (value 1), is the search target, and again it is the first card

dealt, since the search deck's top card, $2\spadesuit$, already has value 1. The new search deck is $4\spadesuit A\spadesuit 3\spadesuit 2\spadesuit$, and the keystream output is $4\clubsuit$. The drive deck is now empty and round 2 is complete.

The second round boundary is handled the same way. The discard pile holds $4\heartsuit 3\heartsuit A\heartsuit 2\heartsuit$, top to bottom, the hearts spent from the drive deck in the order they were played. At $r = 3$ the count cut moves the top three to the bottom, giving $2\heartsuit 4\heartsuit 3\heartsuit A\heartsuit$. The old search deck, $4\spadesuit A\spadesuit 3\spadesuit 2\spadesuit$, becomes the new drive deck; the rotated discard becomes the new search deck; the discard pile is emptied. The suits have crossed back, so the search deck is hearts again and the drive deck is spades again, completing the exchange begun at the first boundary.

The pair of decks recorded at each boundary is the cipher's state, and F is the map from one boundary state to the next. The first boundary above gave C_1 , state code 254; the second gives $C_2 = F(C_1)$, the pair (search $2\heartsuit 4\heartsuit 3\heartsuit A\heartsuit$, drive $4\spadesuit A\spadesuit 3\spadesuit 2\spadesuit$), state code 283. The keystream is the separate sequence of cards read off the top of the search deck, here $2\heartsuit 3\heartsuit 2\spadesuit 4\spadesuit$. The state is what F moves; the keystream is what the cipher emits along the way. One caveat on this walkthrough: it begins from two decks placed in order by hand rather than from a key, so it illustrates the mechanics of the round rather than the trajectory of any particular key. A key arrives at its own starting arrangement through the key schedule described earlier in this chapter.

Two rounds of this four-card example produce four keystream outputs, $2\heartsuit 3\heartsuit 2\spadesuit 4\spadesuit$, or values 1 2 1 3. Each round needed exactly two turns to exhaust the four-card drive deck, confirming by hand the $n / 2 = 2$ outputs per round that the keystream output rate below is built on, and the suit crossover at the round boundary is the clearest physical picture of why the cipher's diffusion does not depend on any single card staying in one deck for long.

The Rotation Variants

The rotation r applied to the discard pile at each round boundary is the second variant parameter studied here. Its origin is worth recording. The value was originally a fixed 5, chosen as an experiment to see whether disturbing the discard pile would lengthen cycles. The constant did not scale: at deck size 4 a rotation of five cards through a four-card pile wrote past the end of the array and corrupted the drive deck, which was corrected by reducing the rotation mod the deck size. The fix raised the design question of whether the value should be fixed, scaled with the deck, or treated as an experimental parameter in its own right, and the last of these was chosen. This work examines rotation values of $r = 0, 1, 3$ and 5. That set should be read as a sample of the parameter rather than an enumeration of it, and its composition is historical: r began as a fixed 5, and 0, 1 and 3 were added around it. It covers three of the four possible rotations at deck size 4, since $5 \bmod 4 = 1$, four of six at deck size 6, and four of eight at deck size 8. Two of the values have a natural physical reading worth noting: $r = 1$ is a single card cut from the top of the discard pile to the bottom, and $r = n - 1$ is its reverse, a single card cut from the bottom to the top, which is $r = 3$ at deck size 4 and $r = 5$ at deck size 6. At deck size 8 the reverse single card cut, $r = 7$, was not among the values run; Chapter 6 records completing the sweep as future work.

One consequence of the modular reduction should be noted before the results, because it explains an apparent coincidence in Chapter 4: at deck size 4, rotation 5 is identical to rotation 1, since $5 \bmod 4 = 1$. The two configurations produce byte-identical output, and this serves as a useful check that the modular correction behaves as intended.

Keystream Output Rate

Comparing deck sizes to one another, or this cipher to a published stream cipher, requires a common unit. Cycle lengths in rounds are not comparable across deck sizes, because a round means something different at each n . The natural unit is keystream bits.

The conversion follows from the structure of the round. Each turn emits one card from an alphabet of n symbols, carrying $\log_2 n$ bits if the output is uniformly distributed, an assumption examined at the end of this section, and each round comprises $n / 2$ turns. A round therefore emits $(n / 2) \times \log_2 n$ keystream bits, and a cycle of length L rounds emits $L \times (n / 2) \times \log_2 n$ keystream bits before the state repeats. Table 1 gives the rate for the deck sizes studied and for the full-scale cipher. The value for $n = 26$ provides an independent check on the derivation: 13 turns per round is exactly what the original implementation's own documentation states for the full-size cipher (in its terms, 13 rounds per full round), and it was confirmed independently by working the cipher through by hand, where a four-card drive deck was observed to exhaust in exactly two turns.

It is worth being explicit about what this conversion is, because the word bits can suggest a step that the cipher does not perform. The cipher never produces bits. Each turn it produces one card, a value in the range 0 to $n - 1$, and the keystream is the sequence of those values.

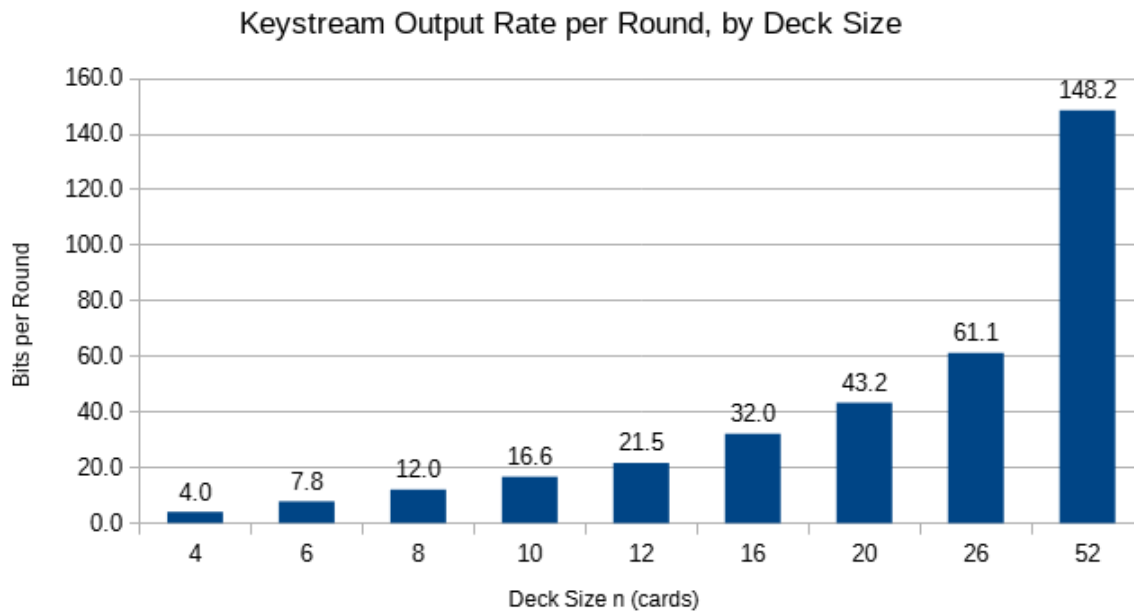
Encryption combines each keystream value with one plaintext letter in the n -letter alphabet by addition mod n , in the same way that Solitaire combines its output with plaintext by addition mod 26, and decryption reverses that step. There is no binary encoding anywhere in the method, and an operator working with physical cards never converts anything into bits. The bit figures quoted throughout this work are a unit of measurement applied to the output afterwards, not a stage of the algorithm.

What $\log_2 n$ measures is the information carried by one symbol drawn uniformly from an alphabet of n symbols, and it follows from a counting argument. A keystream of L outputs, each of which may take any of n values, can take n^L distinct values in total, and distinguishing n^L possibilities requires $\log_2(n^L) = L \times \log_2 n$ binary digits. One output therefore accounts for $\log_2 n$ bits, which is also the Shannon entropy of a uniform distribution over n outcomes. At deck size 8 the figure is exact and even literal: the eight card values correspond one for one with the eight three-bit patterns 000 through 111, so three bits per output could be written down directly. At deck size 6 it is neither, because $\log_2 6 = 2.585$ is not a whole number, and no whole number of bits holds one of six values exactly. That fractional value is the clearest sign that the quantity being counted is information rather than storage. The unit exists so that cycle lengths at different deck sizes can be compared with one another, and with published stream ciphers.

Table 1. Keystream output rate by deck size. Each turn emits exactly one output, so the turns and outputs columns carry the same value.

Deck size n	Turns per round	Outputs per round	Bits per output	Bits per round
4	2	2	2.000	4.00
6	3	3	2.585	7.75
8	4	4	3.000	12.00
26 (full scale)	13	13	4.700	61.11
52	26	26	5.700	148.21

Figure 1. Keystream output rate per round as a function of deck size. The rate grows as $(n / 2) \log_2 n$, so the full-scale 26-card cipher emits roughly five times as many bits per round as the deck-8 variant analyzed here.



One caveat applies to this conversion throughout. Treating each output as carrying $\log_2 n$ bits assumes the output symbol is uniformly distributed. That assumption is not tested anywhere in this work; measuring it is the first item of future work in Chapter 6. Where this work quotes a keystream length in bits, it is therefore an upper bound on the entropy delivered, and should be read as a measure of period rather than of strength.

CHAPTER 4. APPROACH AND ANALYSIS OF THE STATE SPACE

The State Space

The cipher's state at a round boundary is the ordered pair of the two decks. Each deck is a permutation of n distinct cards, so each has $n!$ possible orderings, and the pair has $(n!)^2$. To index the state space, each permutation is converted to its Lehmer code, the rank of that ordering in lexicographic order, which is one-to-one between permutations and the integers 0 through $n! - 1$.

To clarify, lexicographic order is alphabetical order applied to whole arrangements rather than to words: imagine listing every possible ordering of the deck the way a dictionary lists words, with the arrangement that begins with the lowest card first, ties broken by the second card, then the third, and so on. The rank of an arrangement is simply its position in that list, counting from zero. For a four-card deck ABCD is rank 0, ABDC is rank 1, and DCBA, the last of the 24 arrangements, is rank 23.

The Lehmer code is the method for computing that rank directly from an arrangement, without writing the list out: reading the cards left to right, it records for each card how many of the cards not yet used rank below it, and those counts, weighted by factorials, sum to the position number. What matters for this analysis is only the result. Every arrangement of the deck receives its own number, no two arrangements share a number, and any number can be turned back into the arrangement it came from. That reversibility is what allows a permutation to be used as an array index.

The two ranks combine into a single integer:

$$\text{code} = \text{lehmer}(\text{search deck}) \times n! + \text{lehmer}(\text{drive deck})$$

This is one-to-one between states and the integers 0 through $(n!)^2 - 1$, which is what allows the state space to be treated as a flat array. At deck size 8 the codes run from 0 to 1,625,702,399. Table 2 gives the sizes involved.

Table 2. State space and key space by deck size.

Deck size n	State space $(n!)^2$	Key space n^n	Keys per state	Status in this work
4	576	256	0.444	Boundary validation case
6	518,400	46,656	0.090	Primary small-deck case
8	1,625,702,400	16,777,216	0.010	Primary result
10	1.32×10^{13}	10,000,000,000	0.0008	Infeasible; out of scope
26 (full scale)	$(26!)^2 \approx 1.63 \times 10^{53}$	$26^{26} \approx 6.16 \times 10^{36}$	$\sim 3.8 \times 10^{-17}$	Far beyond listing

As the deck grows, the key space falls away from the state space rapidly: at deck size 4 there are more than two states per key, while at deck size 8 there are roughly a hundred states per key. Keys can therefore reach only a shrinking fraction of the state space, and by deck size 10 the disparity is extreme. This is a counting argument rather than a property of the key schedule: with 16,777,216 keys and 1,625,702,400 states at deck size 8, at least 98.97 percent of states cannot be the starting state of any key, however well the schedule behaves. Deck size 4 is small enough to show the effect directly, where all 256 keys together land on 165 of the 576 states and leave 411 that no key starts on. Unreachable here refers to starting states: a key that lands on a cycle goes on to visit every state on it, so an unreachable cycle is one that no key starts on. Deck size 10 was scoped out of this project: at roughly 1.3×10^{13} states it is some 8,100 times larger than deck size 8, beyond both the hardware and the time available for this analysis.

Note on Key Space

The n^n figure used throughout this paper reflects a deliberate experimental choice to set key length equal to deck size. The cipher does not require this. Key length is unconstrained, since a length- L key over an n -symbol alphabet yields n^L , and the IV extends the effective key material further, since the keying string is constructed as $iv + key + iv$. $L = n$ was chosen to keep exhaustive enumeration tractable and to reduce the experiment to a single scaling variable. Reported key-space statistics are therefore properties of this key set, not of the cipher itself.

The Round Map Is One-to-One

Write F for the map that takes one boundary state to the next, that is, the effect of one complete round. The central structural fact underpinning this analysis is that F is one-to-one on the $(n!)^2$ states.

One-to-one means simply that no two different states can lead to the same next state, so the map can be run backwards as well as forwards. That follows here because every individual step can be undone. A count cut is a rotation, which is undone by rotating back. A search cut deals cards in a fixed way until a chosen card appears and then reassembles the piles by a fixed rule, so given the resulting deck and the card that was searched for, the steps can be run backwards to recover the original ordering. The deck exchange at the boundary is just a renaming of which deck is which, and the discard rotation is another rotation. F is a chain of steps that can each be undone, applied to a finite set of states, and is therefore one-to-one.

The consequence is what makes the rest of this chapter possible. A one-to-one map on a finite set has no dead ends: every state lies on exactly one cycle, and the state space splits into separate cycles whose lengths sum exactly to the size of the space. Walking forward from any state is guaranteed to return to that state, and the number of steps taken is that cycle's length.

There is no tail to detect and no rho shape to disentangle, which is the situation Crowley's non-reversibility result denies to Solitaire. It also supplies a strong self-check, used throughout: if the computed cycle lengths do not sum to exactly $(n!)^2$, something is wrong. That check passes in all twelve configurations reported here.

Three Methods

Three methods were used in succession, each replacing the last as the deck size grew. The first iterates over keys. For each key it runs the key schedule of Chapter 3 to obtain that key's starting deck arrangement, then advances the cipher one round at a time, recording each boundary state in a bitset until a state repeats; the number of rounds elapsed is the length of the cycle that key lands on. It produced the first results in this project, and it has two limitations: it sees only cycles that some key reaches, and it reports each distinct length once, so two different cycles of the same length cannot be told apart in its output. The second abandons keys initially and decomposes the whole space. Every unvisited state begins a forward walk that, because F is one-to-one, necessarily returns to its starting point, yielding one cycle and its exact length; a second pass then runs each key for one round and records which cycle its first boundary state belongs to. The result is the complete cycle decomposition together with an exact count of how many keys land on each cycle, including the cycles that no key reaches at all. This is the method behind all results reported here for deck sizes 4 and 6.

The second method does not survive the jump to deck size 8 in its natural form. Storing the successor of every state as a 32-bit integer requires about 6.5 GB for that array alone, and the full working set of the original implementation approached 28 GB; the analysis machine for this project was a laptop with roughly 1.8 GB of usable memory. The third method therefore stores no map at all. Two bit arrays are kept: one marks the states already visited, the other marks the

states reachable by some key. At one bit per state each array occupies about 194 MB, under 400 MB in total. The two arrays are plain allocated bit vectors with local get and set helpers rather than a library structure, which keeps the deck-8 engines free of external dependencies; Lemire's cbitset library is used by the per-key program of the first method above, where a visited set is built for each key. The successor of a state is not stored anywhere; it is recomputed on demand by running a round. This trades time for memory, and the trade is affordable because the total work is exactly $(n!)^2$ round evaluations regardless: the decomposition of all 1.6 billion states completes in roughly fifteen minutes per rotation, a reduction in memory of about seventyfold against the stored-map approach.

Correctness and Validation

Two problems were found during this work, both in the analysis code rather than in the cipher itself. The first was an off-by-one. The per-key implementation incremented its round counter before testing whether the new state had been seen before, so the iteration that re-detects the starting state was counted, and every reported cycle length was exactly one too high. The symptom that exposed it was that cycle lengths did not sum to the state space: deck size 6 totalled 518,408 against an expected 518,400, one round too many for each of its eight cycles. At deck size 4 the total was 577: the three cycles that keys reach hold 574 states, and each gained one round; the fourth, a two-state cycle that no key reaches, was missing for the second reason below. Moving the counter increment after the repeat test made every reported length the true period, and the lengths of all the cycles then summed to exactly $(n!)^2$. The state space had always split perfectly; only the lengths were inflated. The second problem was a reporting artifact rather than a defect: the per-key parser reported distinct cycle lengths, which collapses two genuinely different cycles of equal length into one row and omits cycles that no key reaches, so several

apparent discrepancies against $(n!)^2$ in the project's interim documents are artifacts of this rather than counting errors. All figures in this work are true periods and supersede the interim weekly documents where the two disagree; in particular, the frequently quoted deck-6 figures of a 416,190-round dominant cycle covering 80.37 percent of keys are rotation-0 results inflated by the off-by-one, and read correctly as 416,189 rounds and 80.36 percent.

Because three different implementations were used over the course of the project, in C and in Python, results were cross-validated rather than trusted individually. The Python port reproduces the committed C output exactly for every key at deck size 4, all 256 of them, and at deck size 6, all 46,656, matching both the terminating state code and the cycle length. The deck-8 bitset engine reproduces the deck-6 decomposition and its key counts exactly before being trusted on deck 8. Every configuration satisfies the two structural identities: cycle lengths sum to $(n!)^2$, and key counts sum to n^n . Evaluating the statistical quality of the keystream itself was outside the scope of this validation and is the first item of future work in Chapter 6.

Cycle Structure Results

Figure 2 shows the complete cycle spectrum at rotation 0 for each deck size. The vertical axis is logarithmic, which is necessary: at deck size 8 the cycle lengths span from 4 to more than 728 million, over eight orders of magnitude, and any linear presentation renders all but the largest two cycles invisible.

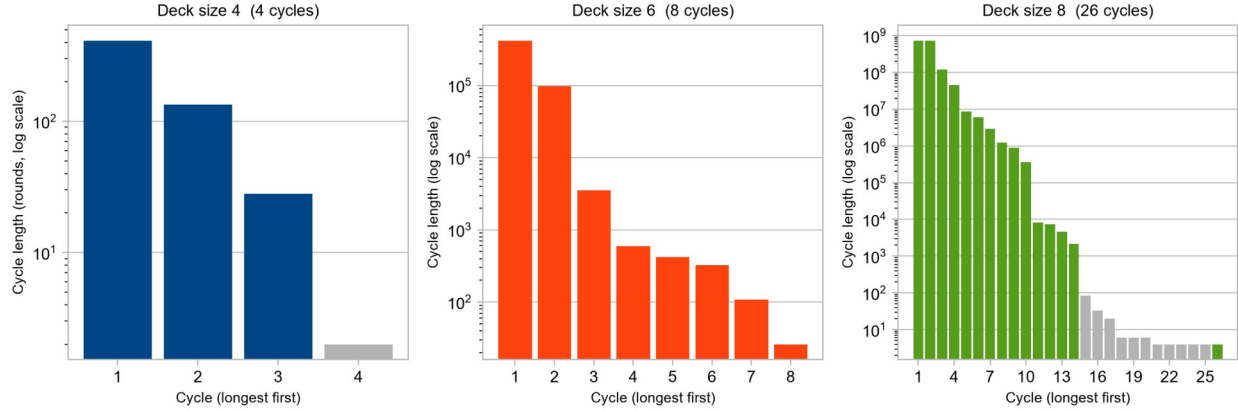


Figure 2. Cycle-length spectrum at rotation 0 for deck sizes 4, 6 and 8, sorted longest first on a logarithmic scale. Grey bars mark cycles that no key's starting state lies on, so no key can ever enter them. The distribution is consistently top-heavy: a small number of very large cycles account for almost the entire state space, followed by a long tail of tiny cycles.

The qualitative shape is the same at every deck size. A handful of very large cycles account for nearly the whole space, followed by a tail of very short cycles. What changes with deck size is the length of that tail: deck size 4 has four cycles in total, deck size 6 has six to eight depending on rotation, and deck size 8 has between 18 and 28. The tail also becomes progressively harder for a key to reach.

Table 3 gives the complete decompositions for deck sizes 4 and 6, which are small enough to list in full.

Table 3. Complete cycle decompositions for deck sizes 4 and 6 (true periods in rounds, longest first). An asterisk marks a cycle that no key's starting state lies on: the key schedule never produces a state on this cycle, so no key can enter it.

Deck	Rotation	Cycle lengths
4	0	412, 134, 28, 2*
4	1	334, 174, 42, 26
4	3	381, 173, 11, 11
4	5	identical to rotation 1 ($5 \bmod 4 = 1$)
6	0	416,189; 97,236; 3,503; 590; 422; 327; 107; 26
6	1	220,024; 128,212; 114,487; 54,589; 969; 53; 53; 13*
6	3	374,262; 96,560; 28,237; 17,634; 1,386; 321
6	5	442,657; 61,407; 14,033; 170; 94; 24; 11; 4*

Deck size 8 is too large to list completely, but Table 4 summarises it, and Figure 3 shows how the rotation reshapes the spectrum.

Table 4. Deck size 8 cycle structure by rotation. The state space is $1,625,702,400 = (8!)^2$ states in every case.

Rotation	Cycles	Key-reachable	Largest cycle	Share of states	Shortest reachable
0	26	15	728,811,383	44.83%	4
1	18	11	780,652,107	48.02%	225
3	26	17	1,068,446,790	65.72%	4
5	28	15	1,274,528,834	78.40%	289

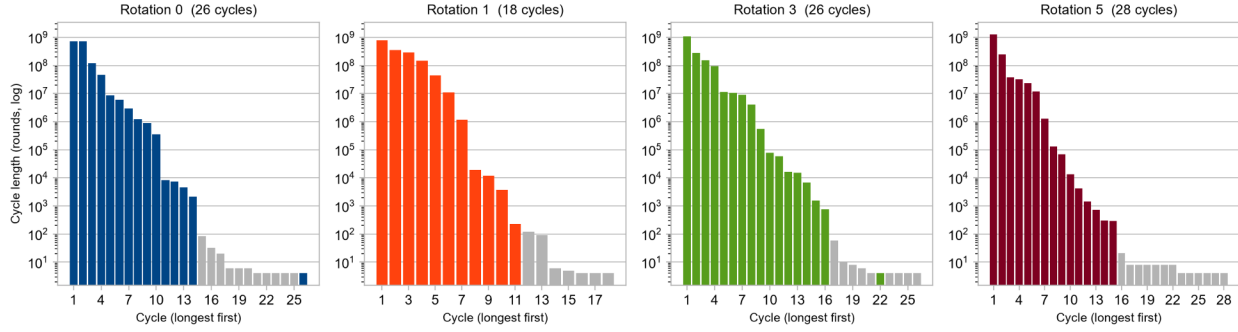


Figure 3. Deck size 8: cycle-length spectrum under each discard rotation, on a logarithmic scale. The rotation is not a cosmetic parameter. It changes the number of cycles from 18 to 28 and the share of the state space held by the largest cycle from 44.83 to 78.40 percent.

The rotation is the striking result of this chapter. It was introduced as an implementation detail, a fixed disturbance of the discard pile with no cryptographic rationale, and yet it substantially determines the cycle structure. At deck size 8 it moves the number of cycles from 18 to 28 and the largest cycle's share of the state space from 44.83 percent to 78.40 percent. More importantly, rotations 0 and 3 leave a cycle of period 4 reachable by a real key, while rotations 1 and 5 keep every reachable cycle at 225 rounds or longer.

Why the rotation has this effect is not established by this work. This is stated as an observation rather than a derivation, and it is the most interesting open question left by this project.

Results in Bits

Table 5 and Figure 4 convert the cycle structure into the common unit established in Chapter 3. Two quantities are given: the longest cycle, which is the best case available to any key, and the expected cycle length weighted by how many keys actually land on each cycle, which is what a randomly chosen key can expect.

Table 5. Keystream period before repetition, in rounds and in bits, by deck size and rotation. Expected values are weighted by the number of keys landing on each cycle.

Deck	Rotation	Expected period (rounds)	Expected period (bits)	Longest cycle (bits)
4	0	328.4	1,314	1,648
4	1	251.8	1,007	1,336
4	3	299.7	1,199	1,524
4	5	251.8	1,007	1,336
6	0	352,645.0	2,734,723	3,227,499
6	1	156,237.7	1,211,606	1,706,261
6	3	290,652.9	2,253,981	2,902,360
6	5	386,367.6	2,996,237	3,432,755
8	0	649,047,078.3	7,788,564,940	8,745,736,596
8	1	517,955,323.3	6,215,463,879	9,367,825,284
8	3	768,688,197.9	9,224,258,375	12,821,361,480
8	5	1,037,714,095.5	12,452,569,146	15,294,346,008

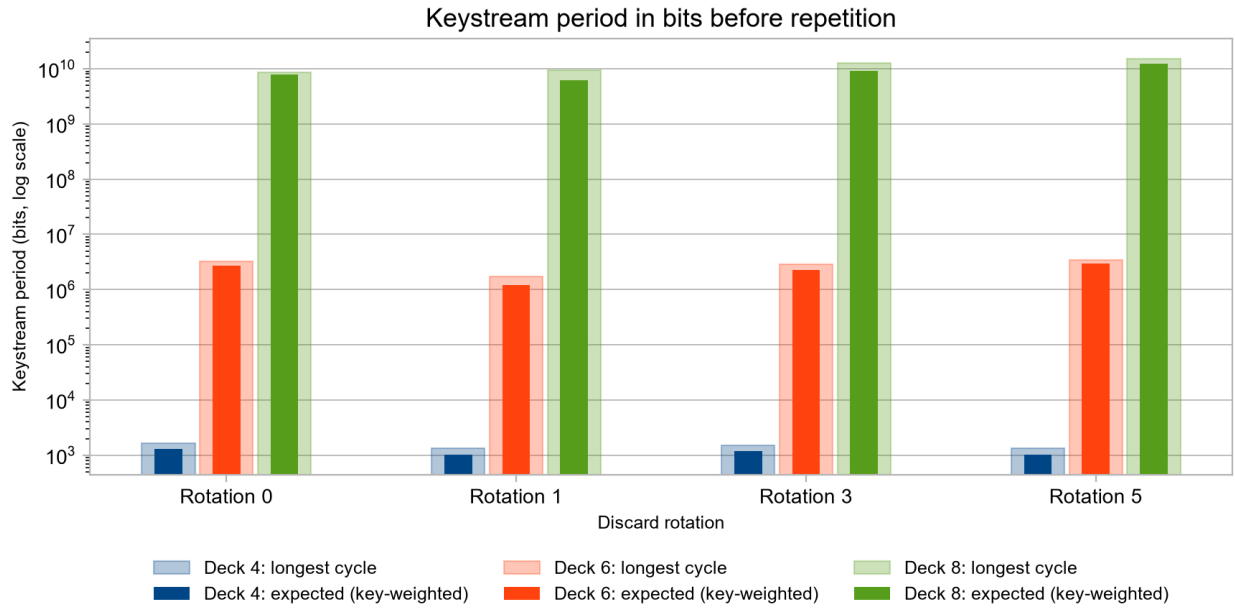


Figure 4. Keystream period in bits before repetition, by deck size and rotation, on a logarithmic scale. Solid bars give the expected period for a key drawn uniformly at random, weighted by the number of keys landing on each cycle; the paler bars beside them give the longest cycle available. The two are close at every deck size, because most keys land on the dominant cycle.

The headline figure is that the strongest deck-8 variant, rotation 5, runs about 1.038×10^9 rounds for a typical key, which at twelve bits per round is about 1.245×10^{10} keystream bits. It is worth keeping the two units apart. The period in rounds cannot exceed the $(8!)^2 = 1.6 \times 10^9$ states available, and at 1.038×10^9 it sits just below that ceiling; the figure in bits is twelve times larger simply because each round emits twelve bits of output. The growth with deck size is steep: from about a thousand bits at deck size 4, to roughly 3×10^6 at deck size 6, to between 6×10^9 and 1.2×10^{10} at deck size 8, an increase of about three and a half orders of magnitude per two cards added to each deck. This is the expected consequence of a state space growing as $(n!)^2$ while a one-to-one map's largest cycle covers a roughly constant fraction of it.

Two observations follow. First, the expected period and the longest cycle are close at every configuration, which is a restatement of the fact that most keys land on the dominant cycle; the gap between them widens only where the structure is fragmented, most visibly at deck size 8 rotation 1, where the largest cycle holds under half the keys. Second, none of these periods is large by modern standards. About 1.2×10^{10} bits is roughly 1.5 gigabytes of keystream, which is ample for the message lengths a hand cipher realistically encrypts, but it is a reminder that the deck sizes analyzed here are research instruments rather than deployable configurations.

A rough projection to a full-scale 26-card deck can be made from these three data points, and is worth stating because it was an explicit question during the work. Across the twelve configurations the expected cycle length holds a broadly comparable fraction of the state space, ranging from about 0.30 to 0.75 depending on deck size and rotation and showing no clear trend with n . Carrying that range forward against a state space of $(26!)^2 \approx 1.63 \times 10^{53}$ gives a period on the order of 10^{54} keystream symbols, roughly 2^{182} bits.

CHAPTER 5. ANALYSIS OF KEYING

Chapter 4 established what cycles exist. This chapter asks a different question: which of the cycles a key can reach, and how the key space distributes over them. The two questions are separate because the key space and the state space are different sizes and the key schedule is the only bridge between them.

The Key Space and the State Space

A key of length n over an alphabet of n symbols gives n^n possible keys, against $(n!)^2$ states. These grow at very different rates. At deck size 4 there are 256 keys and 576 states; at deck size 8 there are 16,777,216 keys and 1,625,702,400 states, so keys are outnumbered roughly a hundred to one. No key schedule can make 16.8 million keys cover 1.6 billion states, so some states are unreachable by construction, and the question is not whether coverage is complete but whether it is unbiased.

The keying procedure does not choose a state directly. It maps each key character to a card value and uses those values to drive the cipher's own count cuts and search cuts, arriving at a starting configuration. The key's first boundary state, written C_1 , is what determines its fate: everything about that key's keystream period follows from which cycle C_1 happens to lie on.

Keys Are Distributed in Proportion to Cycle Length

The cycles listed in Chapter 4 are wildly unequal in size. At deck size 8 under rotation 5, the largest holds more than three quarters of all the states and the smallest holds four. This section asks one plain question about that: does the key schedule treat those cycles evenly? If a cycle covers a tenth of the state space, does it collect a tenth of the keys, or does the schedule favor some regions of the space over others? The answer matters in both directions. A schedule

with favorites would narrow the search an attacker has to make, and it would also place more keys than expected onto whatever periods the favored regions happen to have, long or short.

No such favoritism is detectable. At deck size 8 under rotation 5, the largest cycle covers 78.40 percent of the states and collects 78.40 percent of the keys; the second largest covers 14.98 percent and collects 14.98 percent. The match is not confined to the large cycles: one cycle a thousandth the size of the largest, at 1,251,128 states, collects 12,886 keys where proportionality predicts 12,912. Keys land as though they had been scattered over the state space at random, and they do so across cycles spanning six orders of magnitude in size. The rest of this section measures how closely that holds, and reports the one configuration where the agreement is less than perfect.

Stated precisely: if the key schedule behaved like a uniformly random choice of starting state, the share of keys landing on a given cycle would equal that cycle's share of the state space. Across all four rotations at deck size 8, the largest gap between a cycle's share of the key space and its share of the state space is between 0.005 and 0.025 of a percentage point. Chi-square tests on the well-sampled cycles, meaning those whose expected key count is at least five, give 14.6 on 13 degrees of freedom at rotation 0 ($p = 0.33$), 4.0 on 9 at rotation 1 ($p = 0.91$), 30.5 on 15 at rotation 3 ($p = 0.010$), and 12.0 on 12 at rotation 5 ($p = 0.44$). Three of the four are comfortably consistent with uniform random placement. Rotation 3 is the exception, and it is reported rather than omitted. Two things temper it. With 16,777,216 keys the test has enough power that a very small departure registers, and rotation 3 sits at the top of the range just quoted, at 0.025 of a percentage point, which is the largest gap in the study but negligible in absolute terms; and one marginal result among four tests is weak evidence on its own. No mechanism for it is known, and it is recorded here as a loose end rather than as a finding.

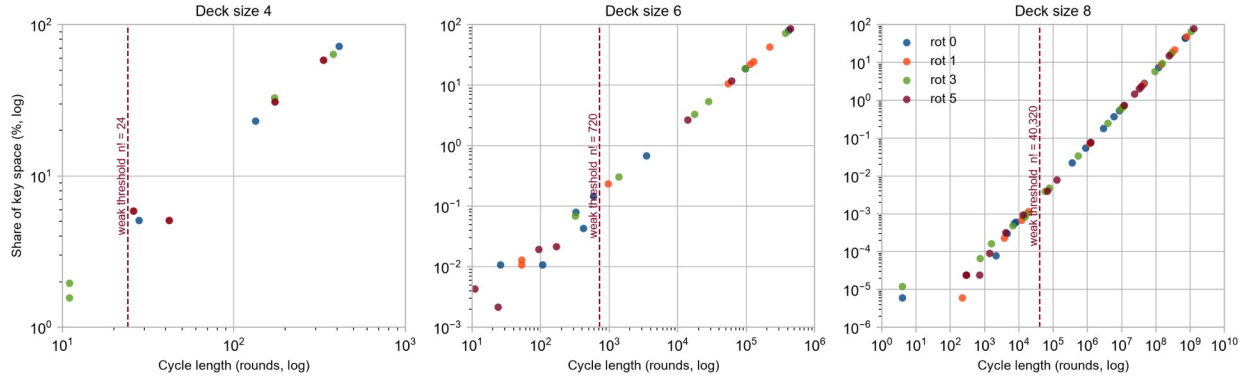


Figure 5. Share of the key space landing on each cycle, plotted against that cycle's length, on logarithmic axes. Points fall along a line of slope one, showing that keys are distributed in proportion to cycle length. The dashed line marks the weak-key threshold $n!$, below which a cycle is considered cryptographically inadequate.

Figure 5 shows this directly. Plotted on logarithmic axes, the share of keys reaching a cycle against that cycle's length falls along a line of slope one across more than eight orders of magnitude and all three deck sizes. The keying step scatters keys evenly across the state space; there is no clustering, no favored region, and no structure for an attacker to exploit in the key schedule itself. This is the strongest positive result in this work. It also accounts for a coincidence in Table 5: averaging cycle length over keys and averaging it over states give answers that differ by under 0.01 percent at every deck-8 rotation. Those are different quantities, and they coincide only because keys are placed in proportion to cycle length. The expected periods in Table 5 are the key-weighted ones, since what a user of the cipher chooses is a key.

It also explains why so many cycles are unreachable. The expected number of keys landing on a cycle of length L at deck size 8 is $n^n L / (n!)^2$, about $0.0103 L$, which falls below one for any cycle shorter than roughly 97 states. The short cycles are therefore almost never reached, not because the cipher avoids them but because uniform placement makes landing on them improbable. That is a reassuring mechanism, but it is a probabilistic one rather than a guarantee, which matters for the next section. An earlier conjecture in this project, that the cycles not reached by any key would together form a single leftover cycle, held at deck sizes 4 and 6 but

fails at deck size 8, where every rotation leaves a scatter of unreachable cycles; it was a small-deck coincidence rather than a structural property.

Weak Keys

A key is called weak here if its first boundary state lies on a cycle shorter than $n!$. The threshold began as a fixed 1,000 rounds, which did not scale across deck sizes, and was replaced by $n!$, giving 24 at deck size 4, 720 at deck size 6 and 40,320 at deck size 8. The rationale is that a keystream repeating within $n!$ rounds is short enough that an adversary who can recognise the repetition can exploit it.

Weakness here is geometric rather than a property of the key schedule. A weak key is not weak because of any pattern in its characters; it is weak because its starting state happens to sit on one of F's small fixed cycles, and every state on that cycle shares its short period. This was confirmed directly at deck size 6, where all five keys producing the shortest cycle lie on the same 26-state loop, their starting positions being rotations of one another.

Table 6. Weak keys by deck size and rotation, using the threshold $n!$.

Deck	Rotation	Weak key share	Approx. frequency	Shortest reachable cycle
4	0	0.0000%	none	28
4	1	0.0000%	none	26
4	3	3.5156%	1 in 28	11
4	5	0.0000%	none	26
6	0	0.2894%	1 in 346	26
6	1	0.0236%	1 in 4,241	53
6	3	0.0686%	1 in 1,458	321
6	5	0.0472%	1 in 2,119	11
8	0	0.0015%	1 in 65,028	4
8	1	0.0020%	1 in 48,913	225
8	3	0.0025%	1 in 40,041	4
8	5	0.0014%	1 in 71,392	289

The proportion of weak keys is small at deck size 8 under every rotation, between 0.0014 and 0.0025 percent, so upwards of 99.997 percent of keys produce a cycle of at least 40,320 rounds. But the aggregate proportion is the wrong figure to reassure oneself with. The final column of Table 6 is the one that matters. Under rotations 0 and 3, a real key reaches a cycle of period four: the entire keystream for that key repeats after four rounds, 48 bits at deck size 8. Exactly one key in 16.8 million lands on it at rotation 0, and two do at rotation 3; for those keys the outcome is catastrophic rather than merely weak. Under rotations 1 and 5 the shortest reachable cycle is 225 and 289 rounds respectively.

This asymmetry is the practical argument for a key check. Any deployment should reject keys whose starting state lies on a short cycle before encryption begins. Whether an operator working by hand could perform such a check is a genuinely open and practically important question: it would require recognising a short cycle from the deck state alone, and if it can be done it would be a useful result in its own right.

Expected Cycle Length and Entropy

Two summary statistics make the rotations comparable. The expected cycle length is the key-weighted mean, the average period a uniformly chosen key obtains. The cycle-spread entropy treats the distribution of keys across cycles as a probability distribution and computes the usual Shannon entropy over it.

Table 7. Keying statistics by deck size and rotation. Entropy is measured over the distribution of keys across cycles.

Deck	Rotation	Key-reachable cycles	Expected cycle length	Entropy (bits)	Keys on largest cycle
4	0	3 of 4	328.4	1.0488	71.88%
4	1	4 of 4	251.8	1.4361	58.20%
4	3	4 of 4	299.7	1.1468	63.67%
4	5	4 of 4	251.8	1.4361	58.20%
6	0	8 of 8	352,645.0	0.7839	80.36%
6	1	7 of 8	156,237.7	1.8684	42.66%
6	3	6 of 6	290,652.9	1.2120	72.28%
6	5	7 of 8	386,367.6	0.6999	85.57%
8	0	15 of 26	649,047,078.3	1.5636	44.83%
8	1	11 of 18	517,955,323.3	1.9383	48.03%
8	3	17 of 26	768,688,197.9	1.5563	65.73%
8	5	15 of 28	1,037,714,095.5	1.0758	78.40%

The entropy figure requires care, because the intuitive reading of it is backwards here.

High entropy means keys are spread across many distinct cycles. That sounds desirable and is not. The ideal outcome for this cipher is that essentially every key lands on one enormous cycle, because that is the outcome in which every key gets the longest available period; spreading keys across several mid-sized cycles simply means a larger share of keys get a shorter, more predictable period. High entropy by this measure is a warning sign rather than a merit.

The two statistics consequently pull in opposite directions, and the pattern is consistent across deck sizes 6 and 8. Rotation 5 has the highest expected cycle length and the lowest entropy: it concentrates keys onto one dominant cycle. Rotation 1 has the highest entropy and the lowest expected length: it fragments the space. Deck size 4 is too small to read anything into, with only four cycles and 256 keys, and is included for completeness rather than as evidence.

Choosing a Rotation

Taking the deck-8 results as the basis, since it is the largest case resolved, the four rotations rank as follows.

- Rotations 0 and 3 are ruled out. Each leaves a period-four cycle reachable by an actual key, which is a catastrophic outcome for whichever key finds it, and no advantage elsewhere compensates for that.
- Rotation 5 gives the longest expected period, at about 1.04 billion rounds, the largest dominant cycle at 78.40 percent of keys, the lowest weak-key rate at one key in 71,392, and the highest floor on reachable cycles at 289 rounds. It is the strongest of the four.
- Rotation 1 is the reasonable alternative. Its floor of 225 rounds is comparable, and its weak-key rate of one in 48,913 is of the same order, but it fragments the key space across more cycles and its expected period is half of rotation 5's.

Rotation 5 is therefore the recommended configuration at deck size 8. The ordering does not carry over unchanged to deck size 6, and the difference is worth stating rather than smoothing over: there rotation 5 again gives the highest expected cycle length, but it has the lowest floor of the four, with a shortest key-reachable cycle of 11 rounds against 53 for rotation 1. The two criteria therefore disagree at deck size 6 and agree at deck size 8, which is a caution against treating either deck size as settling the question on its own. This recommendation rests on the two criteria this work can measure, period and weak-key exposure; it is not a claim about keystream randomness, and it does not extend to a full 26-card deck.

CHAPTER 6. CONCLUSIONS AND FUTURE WORK

Summary of Findings

This work analyzed a hand-executable card-shuffle stream cipher at deck sizes small enough to list in full, across three deck sizes (4,6,8) and four values of the discard rotation (0,1,3,5). The principal findings are as follows.

- The cipher's round transition is one-to-one on its $(n!)^2$ state space, so the space splits into separate cycles with no dead ends. Listing the cycles is therefore exact rather than estimated, and the split supplies a self-check that every configuration reported here passes. This distinguishes the cipher from Solitaire, whose update is not reversible.
- The complete cycle decomposition was computed for all twelve configurations, including deck size 8, where 1,625,702,400 states and 16,777,216 keys were resolved by storing one bit per state and recomputing the transition on demand.
- The discard rotation, introduced as an implementation detail with no cryptographic rationale, substantially determines the cycle structure. At deck size 8 it moves the largest cycle's share of the state space from 44.83 to 78.40 percent. Rotation 5 appears to be the strongest of the four based on cycle size.
- The key schedule shows no measurable bias in how it places keys across cycles. This is a statement about key placement, not about the distribution of the keystream symbols, which was not measured. Keys land on cycles in proportion to cycle length, with the largest deviation across all deck-8 rotations amounting to a few hundredths of a percentage point. The many unreachable cycles are a consequence of the key space being far smaller than the state space, which is expected.
- Expressed in keystream bits, the strongest deck-8 configuration delivers about 1.245×10^{10} bits before repetition for a typical key, and the period grows by roughly three and a half orders of magnitude for every two cards added to each deck.

Limitations

The limitations of this work should be stated as plainly as the findings. This is an exploration of the cipher's structure, not a security analysis. Mapping out the cycle structure establishes how long the keystream can run before it repeats and how the key space spreads over those cycles. It does not establish that the keystream looks random, that the cipher resists known-plaintext or chosen-plaintext attack, or that no structural shortcut exists. No such analysis was attempted here, and the honest summary of the cipher's status is that no weaknesses are known, which is a much weaker statement than that none exist.

The results cover deck sizes 4, 6 and 8. Deck size 10 was scoped out as infeasible, being some 8,100 times larger than deck size 8, so the growth trend rests on three points, one of which, deck size 4, is acknowledged to be too small to carry weight. Projection to a full 26-card deck is therefore suggestive rather than established. The rotation results carry a limitation of the same kind. Four values of r were run at each deck size rather than all of them, so the ranking in Chapter 5 compares four rotations and not the rotation parameter as a whole; at deck size 8 half the possible values, including the reverse single card cut $r = 7$, were never measured. Finally, the mechanism by which the rotation reshapes the cycle structure was observed but never derived, which means the recommendation of rotation 5 is empirical, specific to the deck sizes tested, and drawn from a subset of the available rotations.

Future Work

The following items are listed in priority order, from what would most improve confidence in the cipher to what would least.

- Test the quality of the keystream itself. This work measures how long the keystream runs before it repeats, which is a different question from whether it looks random while it runs. The standard test for this class of cipher is Crowley's measurement of how often two consecutive outputs are equal, together with the wider question of how often each pair of consecutive outputs occurs. Both are cheap to run and neither was attempted here. Until they are done, the keystream quality of this cipher is genuinely unknown, and this is the first thing that should be measured next.
- Search for colliding keys, in direct analogy with the RC4 key collision literature. Because the key space at deck size 8 is small enough to list in full, the question of whether two distinct keys reach the same starting state can be settled exactly, where seventeen years of work on RC4 could only narrow it.
- Design and test a key check that an operator can actually perform. Chapter 5 shows that under rotations 0 and 3 a real key reaches a cycle of period four, so any deployment has to be able to reject a key whose starting state lies on a very short cycle before encryption begins. The open question is whether that can be recognised from the deck arrangement alone, since the check has to be as low-technology as the cipher it protects. A negative answer would be as useful as a positive one: it would mean the rotation must be chosen so that no dangerously short cycle is reachable at all, which is the argument made for rotation 5 in Chapter 5.

- Run the remaining rotations. This work measured $r = 0, 1, 3$ and 5 , which is four of the eight possible values at deck size 8 and leaves the reverse single card cut $r = 7$ untested. The deck- 8 engine already takes the rotation as a parameter, so completing the sweep needs no new code, and it would turn the recommendation of rotation 5 from the best of four into the best of eight. The smaller decks show why this matters: at deck size 4 the untested rotation $r = 2$ has both the longest expected cycle of any deck- 4 rotation and the highest weak-key rate measured anywhere in this work, so untested values cannot safely be assumed to be unremarkable.
- Resolve deck size 10 . The growth trend established here rests on three deck sizes, one of which, deck size 4 , is acknowledged to be too small to carry weight, so a fourth point would materially strengthen any projection toward a full deck. Deck size 10 is the next even size, with $13,168,189,440,000$ states, some $8,100$ times deck size 8 . The cost should be stated honestly, because it is not a matter of a larger machine: at one bit per state the two arrays that let deck size 8 run in under 400 MB would need roughly 3 TB, and at the deck- 8 rate a single rotation would take on the order of three months. Deck size 10 therefore calls for a distributed decomposition, or for an approach that never holds the state space in memory at all, and designing one is the contribution rather than the hardware.
- Investigate key schedules that avoid short cycles. This work offers two defences against a key landing on a very short cycle: detect that key afterwards and reject it, or choose a rotation under which no dangerously short cycle is reachable at all. A third is untried, which is to design the key schedule itself, including the IV that wraps the key, so that a short-cycle starting state is never produced in the first place. The tension worth resolving is that the schedule's chief virtue as measured here is that it shows no bias, placing keys across cycles in proportion to cycle length. A schedule that steers away from short cycles is a deliberately biased one, and whether the bias it introduces costs more than the short cycles it removes is an open question.
- Vary the length of the key and the IV. Every result in this work uses a key of exactly n characters and an empty IV, which fixes the key sequence at n characters and the key space at n^n . Neither restriction is inherent to the cipher: the key schedule consumes a string of any length, and a non-empty IV lengthens the key sequence without enlarging the key space. Whether a longer key sequence spreads keys differently across cycles, and whether it can be used to steer a key away from a short cycle without introducing the bias discussed in the item above, is untested.
- Determine the mechanism by which the discard rotation reshapes the cycle structure. This is the deepest open question raised by the work and the one whose answer would most improve the cipher's design rather than merely its description.

Closing

Hand ciphers have historically been analyzed by the only means their scale permits, which is sampling, and the two best-known results in this area, Crowley's bias in Solitaire and his own withdrawal of Mirdek, both came from statistical observation and code-breaking insight rather than from checking every case. The contribution of this work is to show that shrinking the deck until the state space fits in memory turns questions that were previously matters of estimation into matters of computation. The cycle structure of this cipher at deck size 8 is not estimated or bounded; it is known. Whether the properties observed at these deck sizes persist at twenty-six cards is the question that remains.

REFERENCES

- AlFardan, N., Bernstein, D. J., Paterson, K. G., Poettering, B., & Schuldt, J. C. N. (2013). On the security of RC4 in TLS. Proceedings of the 22nd USENIX Security Symposium.
- Chen, J., & Miyaji, A. (2011). How to find short RC4 colliding key pairs. In Information Security (ISC 2011), Lecture Notes in Computer Science 7001 (pp. 32-46). Springer.
- Crowley, P. (2000). Mirdek: A card cipher inspired by Solitaire.
<http://www.ciphergoth.org/crypto/mirdek/>
- Crowley, P. (2001). Problems with Bruce Schneier's Solitaire.
<http://www.ciphergoth.org/crypto/solitaire/>
- Fluhrer, S. R., & McGrew, D. A. (2001). Statistical analysis of the alleged RC4 keystream generator. In Fast Software Encryption (FSE 2000), Lecture Notes in Computer Science 1978. Springer.
- Fluhrer, S., Mantin, I., & Shamir, A. (2001). Weaknesses in the key scheduling algorithm of RC4. In Selected Areas in Cryptography (SAC 2001), Lecture Notes in Computer Science 2259. Springer.
- Jana, A., & Paul, G. (2018). Revisiting RC4 key collision: Faster search algorithm and new 22-byte colliding key pairs. Cryptography and Communications.
<https://eprint.iacr.org/2016/1001>
- Kaminsky, A. (2017). ElsieFour: A low-tech authenticated encryption algorithm for human-to-human communication (Cryptology ePrint Archive Report 2017/339).
<https://eprint.iacr.org/2017/339>
- Lemire, D. (n.d.). cbitset: A simple bitset library in C. <https://github.com/lemire/cbitset>
- Mantin, I., & Shamir, A. (2002). A practical attack on broadcast RC4. In Fast Software Encryption (FSE 2001), Lecture Notes in Computer Science 2355 (pp. 152-164). Springer.
- Matsui, M. (2009). Key collisions of the RC4 stream cipher. In Fast Software Encryption (FSE 2009), Lecture Notes in Computer Science 5665. Springer.
- McKague, M. E. (2005). Design and analysis of RC4-like stream ciphers [Master's thesis, University of Waterloo]. UWSpace. <https://uwspace.uwaterloo.ca/handle/10012/1141>
- Popov, A. (2015). Prohibiting RC4 cipher suites (RFC 7465). Internet Engineering Task Force.
<https://datatracker.ietf.org/doc/html/rfc7465>
- Rubin, M. (2011). John F. Byrne's Chaocipher revealed: An historical and technical appraisal. Cryptologia, 35(4).

- Schneier, B. (1999). The Solitaire encryption algorithm (Version 1.2).
<https://www.schneier.com/academic/solitaire/>
- Shiu, D. (2019). Analysis of Solitaire (arXiv:1909.06300). <https://arxiv.org/abs/1909.06300>
- Stephenson, N. (1999). Cryptonomicon. Avon Books.
- Tounsi, W., Justus, B., Cuppens-Boulahia, N., Cuppens, F., & Garcia-Alfaro, J. (2014). Probabilistic cycle detection for Schneier's Solitaire keystream algorithm. In 2014 IEEE Eighth International Conference on Software Security and Reliability - Companion (SERE-C) (pp. 113-120).
- van Duin, M. (2001, April 24). RC4 with cards [Mailing list post]. cryptography mailing list.
<https://mail-archive.com/cryptography@wasabisystems.com/msg00065.html>
- Vanhoef, M., & Piessens, F. (2015). All your biases belong to us: Breaking RC4 in WPA-TKIP and TLS. Proceedings of the 24th USENIX Security Symposium.
<https://www.rc4nomore.com/>

APPENDIX. SOFTWARE AND DATA

The analysis in this work was produced by the following programs and data files, which are available as supplemental material to the project.

- `weirdleh-exp.c` - the cipher implementation, parameterised by deck size and rotation, with per-key cycle detection.
- `weird_ss8.c` and `weird_ss8_kc.c` - the one-bit-per-state engines used for the deck-8 decomposition and key counts described in Chapter 4.
- `lehmer.c` - the factorial and permutation-ranking routines that convert a deck ordering to its Lehmer code and back, giving the state indexing described in Chapter 4. It is included by all three C programs above.
- `weird_port.py` - a Python port of the cipher, validated to reproduce the C output exactly for all 256 deck-4 keys and all 46,656 deck-6 keys, used for the full state-space decompositions at deck sizes 4 and 6.
- `deck8kc_rot0.txt`, `deck8kc_rot1.txt`, `deck8kc_rot3.txt` and `deck8kc_rot5.txt` - per-cycle lengths and key counts for the deck-8 state space under each rotation.
- Each of the deck-8 data files is self-checking: the cycle lengths sum to $(8!)^2 = 1,625,702,400$ and the key counts sum to $8^8 = 16,777,216$.
- The full cycle and key decomposition for all 4 rotation values at deck size of 8 is listed in the following pages as companion data for this work.
- Anthropic's Claude (Specific Models: Opus 4.8, Opus 5, and Sonnet 5) Large Language Model was employed as a writing and editing aid to assist with structural organization, clarity of expression, and preliminary drafting of background and literature review content, all based on the author's own research and data. The author retains full responsibility for all substantive work: the design and execution of computational experiments, debugging, data analysis and verification, and every final conclusion, judgment, and recommendation. All material generated by the LLM was thoroughly reviewed, revised, and validated by the author to confirm its accuracy and compliance with Iowa State Universities standards of academic integrity and expectations of Iowa State's Graduate College.

Table 8. The Daniels' Cipher - Deck Size 8: Rotation 0 full decomposition.

Cycle Length	% of all Cycles	Keys Producing It	% of All Keys	Classification
4	<0.01%	0	0.00%	N/A
4	<0.01%	0	0.00%	N/A
4	<0.01%	0	0.00%	N/A
4	<0.01%	0	0.00%	N/A
4	<0.01%	0	0.00%	N/A
4	<0.01%	1	<0.01%	Weak
6	<0.01%	0	0.00%	N/A
6	<0.01%	0	0.00%	N/A
6	<0.01%	0	0.00%	N/A
20	<0.01%	0	0.00%	N/A
33	<0.01%	0	0.00%	N/A
84	<0.01%	0	0.00%	N/A
2,157	<0.01%	13	<0.01%	Weak
4,562	<0.01%	50	<0.01%	Weak
7,405	<0.01%	93	<0.01%	Weak
8,249	<0.01%	101	<0.01%	Weak
360,106	0.02%	3,720	0.02%	Moderate
888,270	0.05%	9,130	0.05%	Moderate
1,217,320	0.07%	12,600	0.08%	Moderate
2,941,287	0.18%	30,097	0.18%	Moderate
5,974,709	0.37%	61,753	0.37%	Moderate
8,608,416	0.53%	88,595	0.53%	Moderate
45,773,410	2.82%	472,183	2.81%	Moderate
118,634,915	7.30%	1,223,477	7.29%	Strong
712,470,032	43.83%	7,353,459	43.83%	Strong
728,811,383	44.83%	7,521,944	44.83%	Maximum (Generator)
Total: 1,625,702,400 = 8!^2	100%	16,777,216 = 8^8	100%	

Table 9. The Daniels' Cipher - Deck Size 8: Rotation 1 full decomposition.

Cycle Length	% of all Cycles	Keys Producing It	% of All Keys	Classification
4	<0.01%	0	0.00%	N/A
4	<0.01%	0	0.00%	N/A
4	<0.01%	0	0.00%	N/A
5	<0.01%	0	0.00%	N/A
6	<0.01%	0	0.00%	N/A
90	<0.01%	0	0.00%	N/A
123	<0.01%	0	0.00%	N/A
225	<0.01%	1	<0.01%	Weak
3,764	<0.01%	38	<0.01%	Weak
12,082	<0.01%	113	<0.01%	Weak
19,231	<0.01%	191	<0.01%	Weak
1,164,775	0.07%	12,042	0.07%	Moderate
10,919,748	0.67%	112,730	0.67%	Moderate
43,809,435	2.69%	452,275	2.70%	Moderate
143,768,005	8.84%	1,484,517	8.85%	Strong
294,157,443	18.09%	3,035,472	18.09%	Strong
351,195,349	21.60%	3,621,861	21.59%	Strong
780,652,107	48.02%	8,057,976	48.03%	Maximum (Generator)
Total: 1,625,702,400 = 8!^2	100%	16,777,216 = 8^8	100%	

Table 10. The Daniels' Cipher - Deck Size 8: Rotation 3 full decomposition.

Cycle Length	% of all Cycles	Keys Producing It	% of All Keys	Classification
4	<0.01%	0	0.00%	N/A
4	<0.01%	0	0.00%	N/A
4	<0.01%	0	0.00%	N/A
4	<0.01%	0	0.00%	N/A
4	<0.01%	0	0.00%	N/A
4	<0.01%	2	<0.01%	Weak
6	<0.01%	0	0.00%	N/A
8	<0.01%	0	0.00%	N/A
10	<0.01%	0	0.00%	N/A
58	<0.01%	0	0.00%	N/A
743	<0.01%	11	<0.01%	Weak
1,580	<0.01%	27	<0.01%	Weak
6,672	<0.01%	80	<0.01%	Weak
15,000	<0.01%	137	<0.01%	Weak
16,180	<0.01%	162	<0.01%	Weak
58,158	<0.01%	650	<0.01%	Moderate
77,773	<0.01%	799	<0.01%	Moderate
539,124	0.03%	5,662	0.03%	Moderate
3,967,555	0.24%	41,329	0.25%	Moderate
9,112,251	0.56%	94,187	0.56%	Moderate
10,694,349	0.66%	110,331	0.66%	Moderate
11,517,585	0.71%	118,692	0.71%	Moderate
93,781,758	5.77%	968,766	5.77%	Strong
152,668,279	9.39%	1,576,343	9.40%	Strong
274,798,497	16.90%	2,831,667	16.88%	Strong
1,068,446,790	65.72%	11,028,371	65.73%	Maximum (Generator)
Total: 1,625,702,400 = 8!^2	100%	16,777,216 = 8^8	100%	

Table 11. The Daniels' Cipher - Deck Size 8: Rotation 5 full decomposition.

Cycle Length	% of All Cycles	Keys Producing It	% of All Keys	Classification
4	<0.01%	0	0.00%	N/A
4	<0.01%	0	0.00%	N/A
4	<0.01%	0	0.00%	N/A
4	<0.01%	0	0.00%	N/A
4	<0.01%	0	0.00%	N/A
4	<0.01%	0	0.00%	N/A
8	<0.01%	0	0.00%	N/A
8	<0.01%	0	0.00%	N/A
8	<0.01%	0	0.00%	N/A
8	<0.01%	0	0.00%	N/A
8	<0.01%	0	0.00%	N/A
8	<0.01%	0	0.00%	N/A
21	<0.01%	0	0.00%	N/A
289	<0.01%	4	<0.01%	Weak
298	<0.01%	4	<0.01%	Weak
718	<0.01%	4	<0.01%	Weak
1,407	<0.01%	15	<0.01%	Weak
4,161	<0.01%	53	<0.01%	Weak
13,301	<0.01%	155	<0.01%	Weak
68,653	<0.01%	665	<0.01%	Moderate
129,227	0.01%	1,307	0.01%	Moderate
1,251,128	0.08%	12,886	0.08%	Moderate
11,948,247	0.73%	123,169	0.73%	Moderate
23,776,907	1.46%	245,118	1.46%	Moderate
32,789,090	2.02%	337,624	2.01%	Moderate
37,662,100	2.32%	389,060	2.32%	Moderate
243,527,947	14.98%	2,513,405	14.98%	Strong
1,274,528,834	78.40%	13,153,747	78.40%	Maximum (Generator)
Total: 1,625,702,400 = 8!^2	100%	16,777,216 = 8^8	100%	